Build and Deployment

This document covers how the system is built, packaged, and deployed to target platforms.

Development Setup

Prerequisites

- Python 3.11 or later
- Tkinter (`python3-tk` package on Linux)

Installation

```
git clone https://github.com/YOUR_USERNAME/kiosk-signin-system.git
cd kiosk-signin-system
python -m venv .venv
source .venv/bin/activate    # Linux/macOS
.venv\Scripts\activate       # Windows
pip install -r requirements.txt
```

Running in Development

```
python kiosk_signin/app.py
```

This starts both the kiosk GUI and the web dashboard on port 5000.

Set `DASHBOARD_SECRET_KEY` before running so dashboard sessions survive restarts (otherwise a random key is generated each run and everyone is logged out):

```
export DASHBOARD_SECRET_KEY=some-long-random-string   # Linux/macOS
$env:DASHBOARD_SECRET_KEY="some-long-random-string"    # Windows PowerShell
```

Alternatively, copy `.env.example` to `.env` and set it there -- it's loaded automatically via `python-dotenv` if present, and `.env` is gitignored so it's never committed.

Dependencies

The `requirements.txt` file lists all Python dependencies:

Package	Purpose
Flask	Web dashboard framework

Werkzeug	WSGI toolkit under Flask (pinned version)
xhtml2pdf	PDF export generation
openpyxl	Excel (.xlsx) file generation
Pillow	Image processing for signature rendering
pytest	Test framework
pytest-cov	Test coverage reporting
coverage	Coverage measurement (used by pytest-cov)
tkreload	Live-reload for Tkinter during development
watchdog	File system monitoring (used by tkreload)
click	CLI framework (Flask dependency)
rich	Terminal formatting
waitress	Production WSGI server for the dashboard
Flask-WTF	CSRF protection for dashboard forms
Flask-Limiter	Login rate limiting
python-dotenv	Optional `.env` file loading for local config

PyInstaller Builds

The system is packaged into standalone executables using PyInstaller. A single binary bundles the Python interpreter, all dependencies, and application files.

Build Command

The core PyInstaller command (from the GitHub Actions workflow):

```
pyinstaller --onefile \
   --add-data "dashboard/templates:dashboard/templates" \
   --add-data "dashboard/static:dashboard/static" \
   --add-data "dashboard/app.py:dashboard" \
   --add-data "kiosk_signin/export_util.py:kiosk_signin" \
   --hidden-import flask \
   --hidden-import flask_wtf \
   --hidden-import flask_limiter \
   --hidden-import dotenv \
```

```
  --hidden-import waitress \
  --hidden-import werkzeug \
  --collect-submodules werkzeug \
  kiosk_signin/app.py \
  --name KioskSignIn
```

Key Options

- `--onefile` -- Produces a single executable file rather than a directory of files. At runtime, the executable extracts its contents to a temporary directory.
- `--add-data` -- Bundles non-Python files (HTML templates, CSS, static assets) into the executable. The format is `source:destination` (colon on Linux, semicolon on Windows). Notably, `dashboard/app.py` itself is bundled this way rather than being statically imported -- PyInstaller copies it as a raw file and it's `exec`'d at runtime via a lazy `from dashboard.app import app`, not analysed for imports ahead of time.
- `--hidden-import` -- Because `dashboard/app.py` is bundled as data rather than analysed, **none** of its imports are auto-detected by PyInstaller, no matter how they're written (top-level or lazy). Every package it imports -- `flask`, `flask_wtf`, `flask_limiter`, `dotenv` (the import name for `python-dotenv`), and `waitress` -- has to be listed explicitly, or the bundled `.exe` will start fine but the background dashboard thread will silently fail with `ModuleNotFoundError` the moment it tries to import `dashboard.app` (silently, because it's a daemon thread with no UI to report the crash to -- the kiosk itself keeps running normally). Any new dependency added to `dashboard/app.py` in the future needs the same treatment.
- `--collect-submodules werkzeug` -- Werkzeug (Flask's underlying WSGI library) uses lazy, `__getattr__`-based imports between its own submodules (e.g. `werkzeug.utils` reaching into `werkzeug.wsgi` for `wrap_file`). On a Windows kiosk build this can surface as `ImportError: cannot import name 'wrap_file' from partially initialized module 'werkzeug.wsgi' (most likely due to a circular import)` the moment the bundled `.exe` tries to import `werkzeug.security` for password hashing -- PyInstaller's frozen import machinery can resolve Werkzeug's internal submodule graph in a different order than a normal on-disk install, catching it mid-initialization. `--collect-submodules werkzeug` forces PyInstaller to bundle all of Werkzeug's submodules together up front rather than discovering them piecemeal, which avoids the partial-initialization ordering issue. This is specific to PyInstaller's Windows-frozen import order, so it can only be verified end-to-end on an actual Windows build of the `.exe`, not in CI or a Linux dev environment.

Runtime Path Handling

When running from a PyInstaller bundle, the application detects the frozen state and adjusts paths:

```
if getattr(sys, "frozen", False):
  _base_dir = os.path.dirname(sys.executable)
  _template_dir = os.path.join(sys._MEIPASS, "dashboard", "templates")
```

- `sys.frozen` is set to `True` when running inside a PyInstaller bundle.
- `sys._MEIPASS` points to the temporary directory where the bundle's contents are extracted.
- `sys.executable` points to the actual executable file on disk.

Database Location

In development, the database is stored alongside the source code (`kiosk_signin/signin_data.db`). In a PyInstaller bundle, the internal filesystem is read-only, so the database is stored in a writable location:

- **Windows**: `%LOCALAPPDATA%\Kiosk Sign-In\signin_data.db`
- **Linux**: Same directory as the executable

```
if getattr(sys, "frozen", False):
    _data_dir = os.path.join(
        os.environ.get("LOCALAPPDATA", os.path.dirname(sys.executable)),
        "Kiosk Sign-In",
    )
    os.makedirs(_data_dir, exist_ok=True)
    DB_PATH = os.path.join(_data_dir, "signin_data.db")
```

CI/CD Pipeline

GitHub Actions Workflow

The build workflow (`.github/workflows/build.yml`) runs automatically when a GitHub release is created. It builds executables for two platforms in parallel:

Windows (x64)

```
build-windows:
    runs-on: windows-latest
    steps:
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install -r requirements.txt pyinstaller
      - run: pyinstaller ... --name KioskSignIn-Windows
```

Runs on a standard Windows runner. Produces an `.exe` file. This is the binary used on the kiosk device.

Linux (x64)

```
build-linux-x64:
    runs-on: ubuntu-22.04
```

Built on Ubuntu 22.04 to match Pop!_OS 22.04's glibc version. Requires `python3-tk` and `python3-dev` system packages. Used for development/testing rather than kiosk deployment.

Release Artifacts

Each build job uploads its executable to the GitHub release using `softprops/action-gh-release@v2`. The release page then has two downloadable binaries:

- `KioskSignIn-Windows.exe`
- `KioskSignIn-Linux-x64`

Workflow Trigger

The workflow is triggered by the `release: created` event:

```
on:
 release:
   types: [created]
```

To create a release: go to the GitHub repository > Releases > Draft a new release > Create a tag (e.g. `v1.0.0`) > Publish. The workflow starts automatically and attaches the built binaries.

Target Platform: Windows Kiosk

The primary deployment target is a Windows 10 touchscreen PC (an HP kiosk unit) installed on-site.

Hardware

- Windows 10 PC with a touchscreen
- HP kiosk unit (as currently deployed)

Deployment Steps

1. Download `KioskSignIn-Windows.exe` from the GitHub release.
2. Copy it to the kiosk PC (no installer needed — it's a single-file executable).
3. Run it. The kiosk window opens maximised and the web dashboard starts automatically on port 5000.
4. Optionally configure it to start on login (e.g. a shortcut in the Windows Startup folder, or Task Scheduler) so the kiosk comes up automatically after a reboot.

Network Access

The dashboard is served on `0.0.0.0:5000` by Waitress (a production WSGI server -- see 11-threading-and-concurrency.md), making it accessible from any device on the same network. This allows administrators and staff to access the dashboard from their own computer or phone while the kiosk remains at the sign-in point, and the sign-in/out page is mobile-friendly for that reason.

Internet Access via a Reverse Tunnel

The dashboard can additionally be exposed to the internet at a URL like `https://kiosk-dashboard.example.com` through a tunnel tool (e.g. Cloudflare Tunnel's `cloudflared`, running as a Windows service on the kiosk), which handles HTTPS/TLS termination at its own edge — no certificates, reverse proxy, port forwarding, or static IP need to be managed on site. The tunnel, DNS, routine checks, and troubleshooting are covered in 14-dashboard-hosting.md.

Because of this, the dashboard's own hardening focuses on what's still its responsibility: a real (non-hardcoded) secret key, hashed multi-user logins with per-account lockout, CSRF protection, login rate limiting, and security response headers. Session cookies are *not* marked `Secure` by default, because the same process also serves plain-HTTP LAN access (set `DASHBOARD_COOKIE_SECURE=true` only for an HTTPS-only deployment). See 09-authentication-and-security.md for the full picture.

Deployment note: because the secret key source and session format changed as part of this hardening, every existing browser session is invalidated the first time this version runs -- everyone (including the admin) needs to log in again afterwards. This is expected, not a bug.